

7.0 TRANSMITTING AND RECEIVING PACKETS

7.1 Transmitting Packets

The MAC inside the ENC28J60 will automatically generate the preamble and Start-Of-Frame delimiter fields when transmitting. Additionally, the MAC can generate any padding (if needed) and the CRC if configured to do so. The host controller must generate and write all other frame fields into the buffer memory for transmission.

Additionally, the ENC28J60 requires a single per packet control byte to precede the packet for transmission. The per packet control byte is organized as shown in Figure 7-1. Before transmitting packets, the MAC registers which alter the transmission characteristics should be initialized as documented in **Section 6.0 “Initialization”**.

For an example of how the entire transmit packet and results will look in memory, see Figure 7-2.

FIGURE 7-1: FORMAT FOR PER PACKET CONTROL BYTES

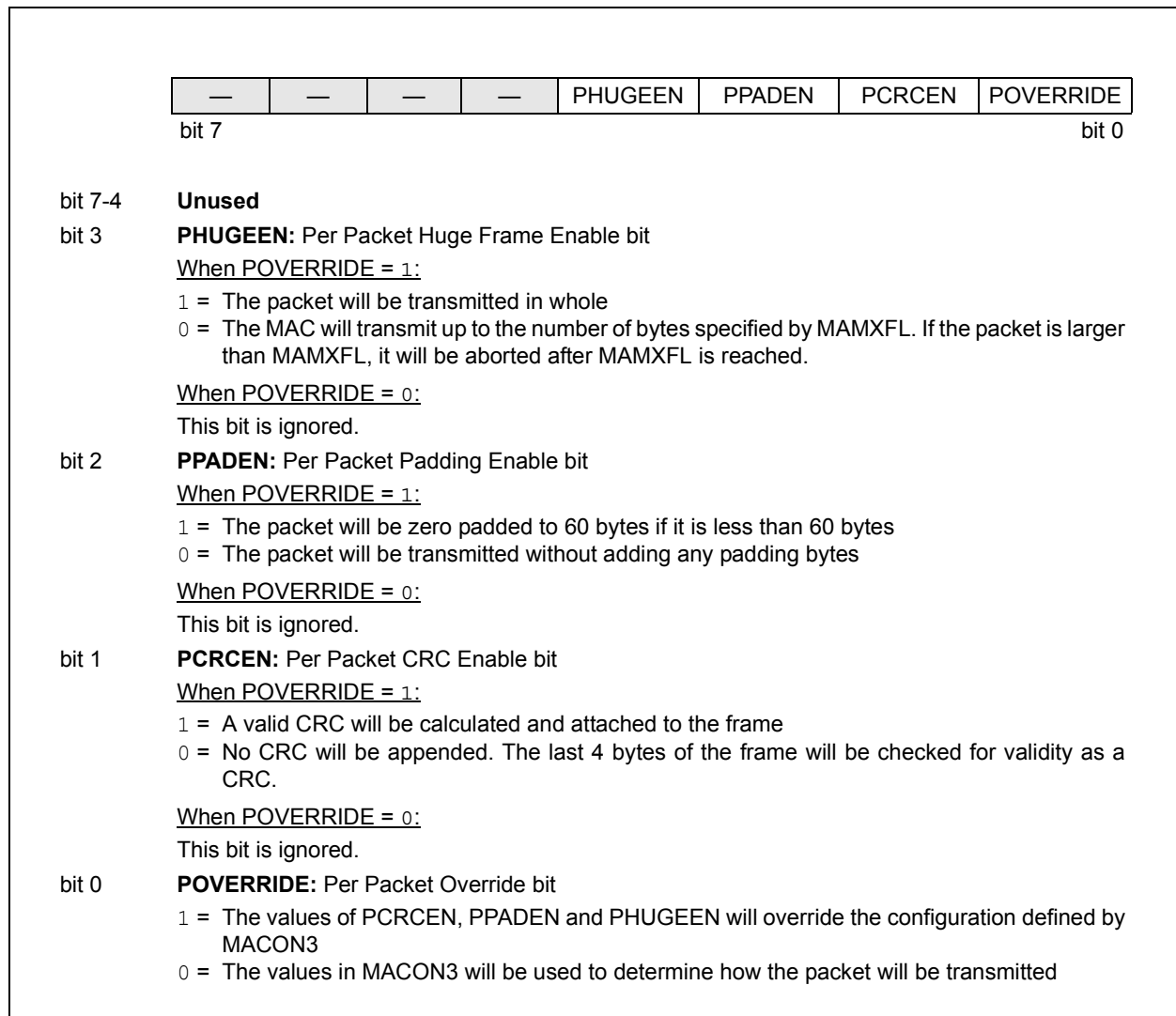
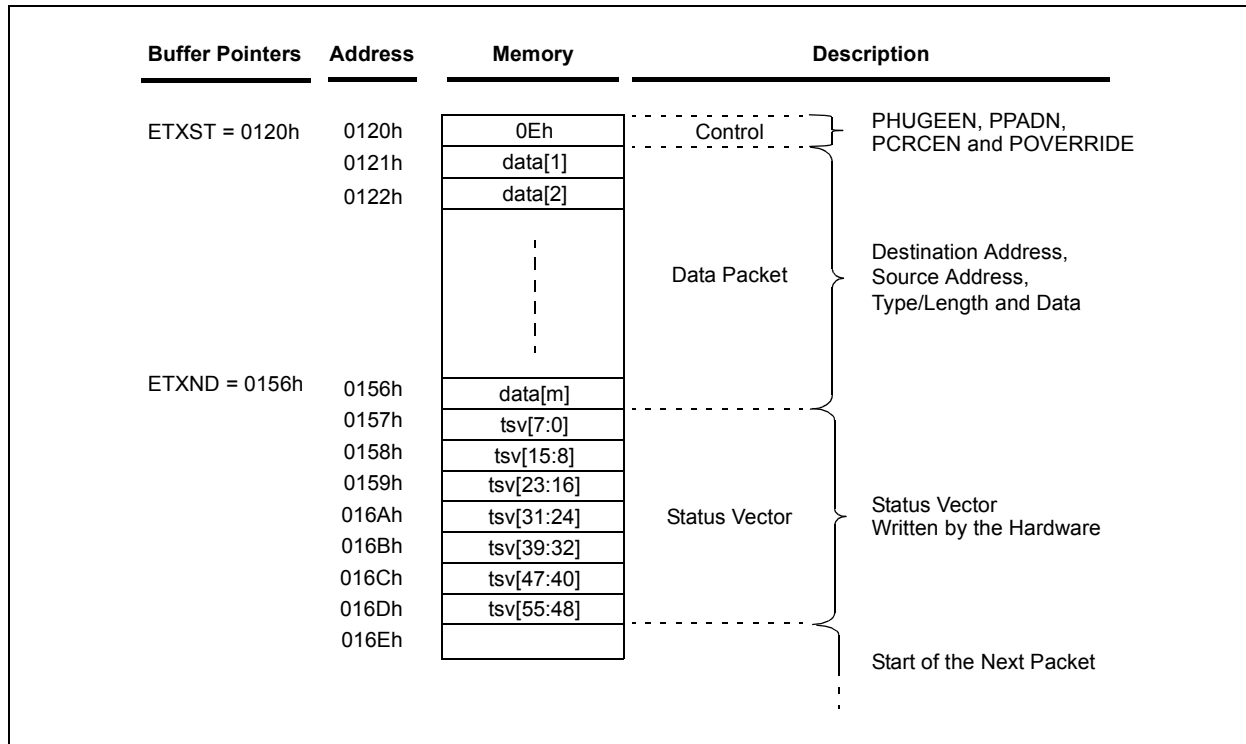


FIGURE 7-2: SAMPLE TRANSMIT PACKET LAYOUT



To achieve the example layout shown in Figure 7-2 and to transmit a packet, the host controller should:

1. Appropriately program the ETXST Pointer to point to an unused location in memory. It will point to the per packet control byte. In the example, it would be programmed to 0120h. It is recommended that an even address be used for ETXST.
2. Use the WBM SPI command to write the per packet control byte, the destination address, the source MAC address, the type/length and the data payload.
3. Appropriately program the ETXND Pointer. It should point to the last byte in the data payload. In the example, it would be programmed to 0156h.
4. Clear EIR.TXIF, set EIE.TXIE and set EIE.INTIE to enable an interrupt when done (if desired).
5. Start the transmission process by setting ECON1.TXRTS.

If a DMA operation was in progress while the TXRTS bit was set, the ENC28J60 will wait until the DMA operation is complete before attempting to transmit the packet. This possible delay is required because the

DMA and transmission engine share the same memory access port. Similarly, if the DMAST bit in ECON1 is set after TXRTS is already set, the DMA will wait until the TXRTS bit becomes clear before doing anything. While the transmission is in progress, none of the unshaded bits (except for the EECON1 register's bits) in Table 7-2 should be changed. Additionally, none of the bytes to be transmitted should be read or written to through the SPI. If the host controller wishes to cancel the transmission, it can clear the TXRTS bit.

When the packet is finished transmitting or is aborted due to an error/cancellation, the ECON1.TXRTS bit will be cleared, a seven-byte transmit status vector will be written to the location pointed to by ETXND + 1, the EIR.TXIF will be set and an interrupt will be generated (if enabled). The ETXST and ETXND Pointers will not be modified. To check if the packet was successfully transmitted, the ESTAT.TXABRT bit should be read. If it was set, the host controller may interrogate the ESTAT.LATECOL bit in addition to the various fields in the transmit status vector to determine the cause. The transmit status vector is organized as shown in Table 7-1. Multi-byte fields are written in little-endian format.

TABLE 7-1: TRANSMIT STATUS VECTORS

Bit	Field	Description
55-52	Zero	0
51	Transmit VLAN Tagged Frame	Frame's length/type field contained 8100h which is the VLAN protocol identifier.
50	Backpressure Applied	Carrier sense method backpressure was previously applied.
49	Transmit Pause Control Frame	The frame transmitted was a control frame with a valid pause opcode.
48	Transmit Control Frame	The frame transmitted was a control frame.
47-32	Total Bytes Transmitted on Wire	Total bytes transmitted on the wire for the current packet, including all bytes from collided attempts.
31	Transmit Underrun	Reserved. This bit will always be '0'.
30	Transmit Giant	Byte count for frame was greater than MAMXFL.
29	Transmit Late Collision	Collision occurred beyond the collision window (MACLCON2).
28	Transmit Excessive Collision	Packet was aborted after the number of collisions exceeded the retransmission maximum (MACLCON1).
27	Transmit Excessive Defer	Packet was deferred in excess of 24,287 bit times (2.4287 ms).
26	Transmit Packet Defer	Packet was deferred for at least one attempt but less than an excessive defer.
25	Transmit Broadcast	Packet's destination address was a Broadcast address.
24	Transmit Multicast	Packet's destination address was a Multicast address.
23	Transmit Done	Transmission of the packet was completed.
22	Transmit Length Out of Range	Indicates that frame type/length field was larger than 1500 bytes (type field).
21	Transmit Length Check Error	Indicates that frame length field value in the packet does not match the actual data byte length and is not a type field. MACON3.FRMLNEN must be set to get this error.
20	Transmit CRC Error	The attached CRC in the packet did not match the internally generated CRC.
19-16	Transmit Collision Count	Number of collisions the current packet incurred during transmission attempts. It applies to successfully transmitted packets and as such, will not show the possible maximum count of 16 collisions.
15-0	Transmit Byte Count	Total bytes in frame not counting collided bytes.

TABLE 7-2: SUMMARY OF REGISTERS USED FOR PACKET TRANSMISSION

Register Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset Values on page
EIE	INTIE	PKTIE	DMAIE	LINKIE	TXIE	r	TXERIE	RXERIE	13
EIR	—	PKTIF	DMAIF	LINKIF	TXIF	r	TXERIF	RXERIF	13
ESTAT	INT	BUFER	r	LATECOL	—	RXBUSY	TXABRT	CLKRDY ⁽¹⁾	13
ECON1	TXRST	RXRST	DMAST	CSUMEN	TXRTS	RXEN	BSEL1	BSEL0	13
ETXSTL	TX Start Low Byte (ETXST<7:0>)								13
ETXSTH	—	—	—	TX Start High Byte (ETXST<12:8>)					13
ETXNDL	TX End Low Byte (ETXND<7:0>)								13
ETXNDH	—	—	—	TX End High Byte (ETXND<12:8>)					13
MACON1	—	—	—	r	TXPAUS	RXPAUS	PASSALL	MARXEN	14
MACON3	PADCFG2	PADCFG1	PADCFG0	TXCRCEN	PHDREN	HFRMEN	FRMLNEN	FULDPX	14
MACON4	—	DEFER	BPEN	NOBKOFF	—	—	r	r	14
MABBIPG	—	Back-to-Back Inter-Packet Gap (BBIPG<6:0>)							14
MAIPGL	—	Non-Back-to-Back Inter-Packet Gap Low Byte (MAIPGL<6:0>)							14
MAIPGH	—	Non-Back-to-Back Inter-Packet Gap High Byte (MAIPGH<6:0>)							14
MACLCON1	—	—	—	—	Retransmission Maximum (RETMAX<3:0>)				14
MACLCON2	—	—	Collision Window (COLWIN<5:0>)						14
MAMXFLL	Maximum Frame Length Low Byte (MAMXFL<7:0>)								14
MAMXFLH	Maximum Frame Length High Byte (MAMXFL<15:8>)								14

Legend: — = unimplemented, r = reserved bit. Shaded cells are not used.

Note 1: CLKRDY resets to '0' on Power-on Reset but is unaffected on all other Resets.

7.2 Receiving Packets

7.2.1 ENABLING RECEPTION

Assuming that the receive buffer has been initialized, the MAC has been properly configured and the receive filters have been configured to receive Ethernet packets, the host controller should:

1. If an interrupt is desired whenever a packet is received, set EIE.PKTIE and EIE.INTIE.
2. If an interrupt is desired whenever a packet is dropped due to insufficient buffer space, clear EIR.RXERIF and set both EIE.RXERIE and EIE.INTIE
3. Enable reception by setting ECON1.RXEN.

After setting RXEN, the Duplex mode and the Receive Buffer Start and End Pointers should not be modified. Additionally, to prevent unexpected packets from arriving, it is recommended that RXEN be cleared before altering the receive filter configuration (ERXFCON) and MAC address.

After reception is enabled, packets which are not filtered out will be written into the circular receive buffer. Any packet which does not meet the necessary filter criteria will be discarded and the host controller will not have any means of identifying that a packet was thrown away. When a packet is accepted and completely written into the buffer, the EPKTCNT register will increment, the EIR.PKTIF bit will be set, an interrupt will be generated (if enabled) and the Hardware Write Pointer, ERXWRPT, will automatically advance.

7.2.2 RECEIVE PACKET LAYOUT

Figure 7-3 shows the layout of a received packet. The packets are preceded by a six-byte header which contains a Next Packet Pointer, in addition to a receive status vector which contains receive statistics, including the packet's size. This receive status vector is shown in Table 7-3.

If the last byte in the packet ends on an odd value address, the hardware will automatically add a padding byte when advancing the Hardware Write Pointer. As such, all packets will start on an even boundary.

FIGURE 7-3: SAMPLE RECEIVE PACKET LAYOUT

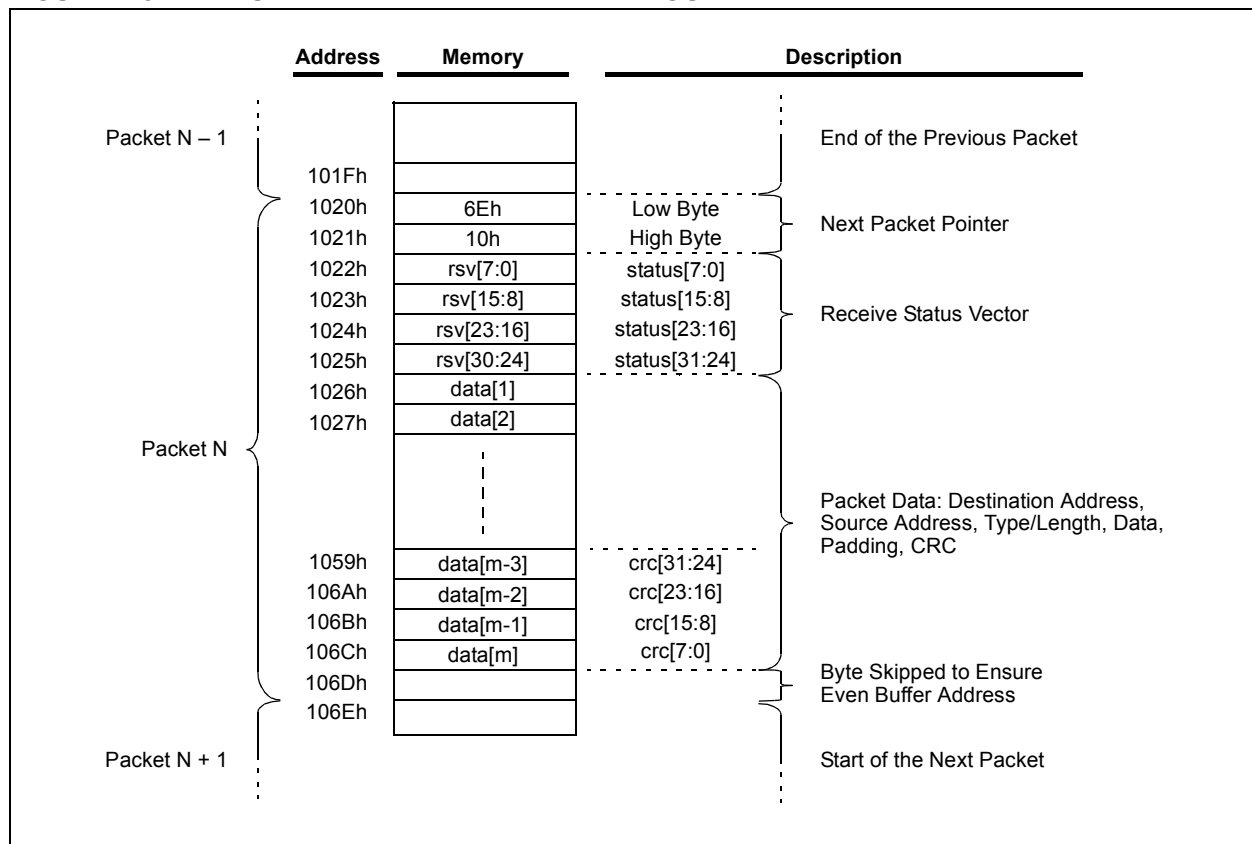


TABLE 7-3: RECEIVE STATUS VECTORS

Bit	Field	Description
31	Zero	0
30	Receive VLAN Type Detected	Current frame was recognized as a VLAN tagged frame.
29	Receive Unknown Opcode	Current frame was recognized as a control frame but it contained an unknown opcode.
28	Receive Pause Control Frame	Current frame was recognized as a control frame containing a valid pause frame opcode and a valid destination address.
27	Receive Control Frame	Current frame was recognized as a control frame for having a valid type/length designating it as a control frame.
26	Dribble Nibble	Indicates that after the end of this packet, an additional 1 to 7 bits were received. The extra bits were thrown away.
25	Receive Broadcast Packet	Indicates packet received had a valid Broadcast address.
24	Receive Multicast Packet	Indicates packet received had a valid Multicast address.
23	Received Ok	Indicates that at the packet had a valid CRC and no symbol errors.
22	Length Out of Range	Indicates that frame type/length field was larger than 1500 bytes (type field).
21	Length Check Error	Indicates that frame length field value in the packet does not match the actual data byte length and specifies a valid length.
20	CRC Error	Indicates that frame CRC field value does not match the CRC calculated by the MAC.
19	Reserved	
18	Carrier Event Previously Seen	Indicates that at some time since the last receive, a carrier event was detected. The carrier event is not associated with this packet. A carrier event is activity on the receive channel that does not result in a packet receive attempt being made.
17	Reserved	
16	Long Event/Drop Event	Indicates a packet over 50,000 bit times occurred or that a packet was dropped since the last receive.
15-0	Received Byte Count	Indicates length of the received frame. This includes the destination address, source address, type/length, data, padding and CRC fields. This field is stored in little-endian format.

7.2.3 READING RECEIVED PACKETS

To process the packet, the host controller will normally use the RBM SPI command and start reading from the beginning of the next Packet Pointer. The host controller will save the next Packet Pointer, any necessary bytes from the receive status vector and then proceed to read the actual packet contents. If ECON2.AUTOINC is set, it will be able to sequentially read the entire packet without ever modifying the ERDPT registers. The Read Pointer would automatically wrap at the end of the circular receive buffer to the beginning.

In the event that the application needed to do random access to the packet, it would be necessary to manually calculate the proper ERDPT, taking care to not exceed the end of the receive buffer if the packet spans the ERXND-to-ERXST buffer boundary. In other words, given the packet start address and a desired offset, the application should follow the logic shown in Example 7-1.

EXAMPLE 7-1: RANDOM ACCESS ADDRESS CALCULATION

```

if Packet Start Address + Offset > ERXND, then
    ERDPT = Packet Start Address + Offset - (ERXND - ERXST + 1)
else
    ERDPT = Packet Start Address + Offset
    
```

7.2.4 FREEING RECEIVE BUFFER SPACE

After the host controller has processed a packet (or part of the packet) and wishes to free the buffer space used by the processed data, the host controller must advance the Receive Buffer Read Pointer, ERXRDPT. The ENC28J60 will always write up to, but not including, the memory pointed to by the Receive Buffer Read Pointer. If the ENC28J60 ever attempts to overwrite the Receive Buffer Read Pointer location, the packet in progress will be aborted, the EIR.RXERIF will be set and an interrupt will be generated (if enabled). In this manner, the hardware will never overwrite unprocessed packets. Normally, the ERXRDPT will be advanced to the value pointed to by the next Packet Pointer which precedes the receive status vector for the current packet. Following such a procedure will not require any pointer calculations to account for wrapping at the end of the circular receive buffer.

The Receive Buffer Read Pointer Low Byte (ERXRDPTL register) is internally buffered to prevent the pointer from moving when only one byte is updated through the SPI. To move ERXRDPT, the host controller must write to ERXRDPTL first. The write will update the internal buffer but will not affect the register. When the host controller writes to ERXRDPTH, the internally buffered low byte will be loaded into the ERXRDPTL register at the same time. The ERXRDPT bytes can be read in any order. When they are read, the actual value of the registers will be returned. As a result, the buffered low byte is not readable.

In addition to advancing the Receive Buffer Read Pointer, after each packet is fully processed, the host controller must write a '1' to the ECON2.PKTDEC bit. Doing so will cause the EPKTCNT register to decrement by 1. After decrementing, if EPKTCNT is '0', the EIR.PKTIF flag will automatically be cleared. Otherwise, it will remain set, indicating that additional packets are in the receive buffer and are waiting to be processed. Attempts to decrement EPKTCNT below 0 are ignored. Additionally, if the EPKTCNT register ever maximizes at 255, all new packets which are received will be aborted, even if buffer space is available. To indicate the error, the EIR.RXERIF will be set and an interrupt will be generated (if enabled). To prevent this condition, the host controller must properly decrement the counter whenever a packet is processed.

Because only one pointer is available to control buffer area ownership, the host controller must process packets in the order they are received. If the host controller wishes to save a packet to be processed later, it should copy the packet to an unused location in memory. It may accomplish this efficiently using the integrated DMA controller (see **Section 13.0 "Direct Memory Access Controller"**).

7.2.5 RECEIVE BUFFER FREE SPACE

At any time the host controller wishes to know how much receive buffer space is remaining, it should read the Hardware Write Pointer (ERXWRPT registers) and compare it with the ERXRDPT registers. Combined with the known size of the receive buffer, the free space can be derived.

Note: The ERXWRPT registers only update when a packet has been successfully received. If the host controller reads it just before another packet is to be successfully completed, the value returned could be stale and off by the maximum frame length permitted (MAMXFL) plus 7. Furthermore, as the host controller reads one byte of ERXWRPT, a new packet may arrive and update the pointer before the host controller has an opportunity to read the other byte of ERXWRPT.

When reading the ERXWRPT register with the receive hardware enabled, special care must be taken to ensure the low and high bytes are read as a matching set.

To be assured that a matching set is obtained:

1. Read the EPKTCNT register and save its contents.
2. Read ERXWRPTL and ERXWRPTH.
3. Read the EPKTCNT register again.
4. Compare the two packet counts. If they are not the same, go back to step 2.

With the Hardware Write Pointer obtained, the free space can be calculated as shown in Example 7-2. The hardware prohibits moving the Write Pointer to the same value occupied by ERXRDPT (except when the Buffer Pointers are being configured), so at least one byte will always go unused in the buffer. The example calculation reflects the lost byte.

EXAMPLE 7-2: RECEIVE BUFFER FREE SPACE CALCULATION

if ERXWRPT > ERXRDPT, then

$$\text{Free Space} = (\text{ERXND} - \text{ERXST}) - (\text{ERXWRPT} - \text{ERXRDPT})$$

else if ERXWRPT = ERXRDPT, then

$$\text{Free Space} = (\text{ERXND} - \text{ERXST})$$

else

$$\text{Free Space} = \text{ERXRDPT} - \text{ERXWRPT} - 1$$

TABLE 7-4: SUMMARY OF REGISTERS USED FOR PACKET RECEPTION

Register Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset Values on page
EIE	INTIE	PKTIE	DMAIE	LINKIE	TXIE	r	TXERIE	RXERIE	13
EIR	—	PKTIF	DMAIF	LINKIF	TXIF	r	TXERIF	RXERIF	13
ESTAT	INT	BUFER	r	LATECOL	—	RXBUSY	TXABRT	CLKRDY ⁽¹⁾	13
ECON2	AUTOINC	PKTDEC	PWRSV	r	VRPS	—	—	—	13
ECON1	TXRST	RXRST	DMAST	CSUMEN	TXRTS	RXEN	BSEL1	BSEL0	13
ERXSTL	RX Start Low Byte (ERXST<7:0>)								13
ERXSTH	—	—	—	RX Start High Byte (ERXST<12:8>)					13
ERXNDL	RX End Low Byte (ERXND<7:0>)								13
ERXNDH	—	—	—	RX End High Byte (ERXND<12:8>)					13
ERXRDP TL	RX RD Pointer Low Byte (ERXRDPT<7:0>)								13
ERXRDP TH	—	—	—	RX RD Pointer High Byte (ERXRDPT<12:8>)					13
ERXFCON	UCEN	ANDOR	CRCEN	PMEN	MPEN	HTEN	MCEN	BCEN	14
EPKTCNT	Ethernet Packet Count								14
MACON1	—	—	—	r	TXPAUS	RXPAUS	PASSALL	MARXEN	14
MACON3	PADCFG2	PADCFG1	PADCFG0	TXCRCEN	PHDREN	HFRMEN	FRMLNEN	FULDPX	14
MAMXFL L	Maximum Frame Length Low Byte (MAMXFL<7:0>)								14
MAMXFL H	Maximum Frame Length High Byte (MAMXFL<15:8>)								14

Legend: — = unimplemented, r = reserved bit. Shaded cells are not used.

Note 1: CLKRDY resets to '0' on Power-on Reset but is unaffected on all other Resets.